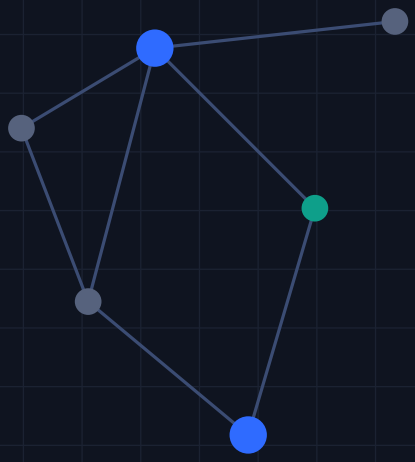
QUICK START

REA

Reverse Engineer Anything



Install REA, connect it to your coding agent,
and run your first investigation in about 20 minutes.

In this guide

1. **Prepare your computer**
   Node.js and a working folder
2. **Choose a coding agent**
   Claude Code, Codex, Cursor and more
3. **Connect REA**
   One setup command, then a health check
4. **Run your first investigation**
   A six-file example app
5. **Inspect a real Electron app**
   Find and analyze its app.asar

Prepared by Tushar (@aibytusharm) · Based on rea.tools/get-started, checked 25 October 2026

REA is open source under the MIT license. Not an official REA publication.

OVERVIEW

What REA does

REA gives your AI coding agent a set of inspection tools for figuring out how software works. You ask a plain question, like *“what happens when I click Export?”*, and your agent uses REA to read the app's files and answer with file and line references.

Two pieces make this work. **MCP** is the connection that lets your agent call REA's tools. A **skill** gives the agent a step-by-step workflow for investigating software. The setup command installs both.

Piece	What it is	Who does what
Coding agent	The AI you chat with (Claude Code, Codex, Cursor...)	Does the reasoning and answers you
REA	Local inspection tools plus an investigation skill	Reads files, maps code, finds connections
Provider (optional)	Ghidra, Hopper or IDA for native programs	Disassembles .exe, .dylib and similar

Best first targets

JavaScript and Electron apps are the easiest place to start. They need nothing beyond Node.js, and REA can analyze them **without running the app**. Many popular desktop apps are built on Electron.

> **Use it responsibly**
> Analyze apps you own, open-source apps whose license allows it, or software you have permission to study. Many commercial apps forbid reverse engineering in their terms of service, even when they are free to use. Never open, decrypt or share other people's data.

1 Prepare your computer

Use the computer where the files you want to inspect are stored.

Open a terminal

- **Windows:** open the Start menu, type *PowerShell*, press Enter.
- **macOS:** press ⌘ Space, type *Terminal*, press Enter.
- **Linux:** press Ctrl Alt T on most desktops.

Check Node.js. REA needs Node.js 22 (22.19 or newer), 24 (24.11 or newer) or 26+. npm comes with it.

TERMINAL
```
node -v
npm -v
```

If the version is older, or you see “command not found”, install Node.js 24 LTS:

System	How to install
Windows	`winget install --exact --id OpenJS.NodeJS.LTS`, then close and reopen PowerShell
macOS	Download the Node.js 24 LTS .pkg from nodejs.org, or run `brew install node`
Linux	Install fnm (`curl -fsSL https://fnm.vercel.app/install | bash`), reopen the terminal, then `fnm install 24 && fnm default 24`

> **Windows tip**
> If PowerShell says *running scripts is disabled on this system* when you run npx, use `npx.cmd` instead of `npx`, and `npm.cmd` instead of `npm`.

Make a working folder and keep this terminal open:

```
TERMINAL
mkdir rea-work
cd rea-work
```

2 Choose a coding agent

REA supplies the tools. Your agent is the AI you talk to.

Pick one. If you already use one of these, skip ahead to Step 3.

Agent	Install	Then
Claude Code	macOS/Linux: `curl -fsSL https://claude.ai/install.sh | bash` Windows: `irm https://claude.ai/install.ps1 | iex`	Run `claude` in your folder and sign in
Codex	macOS/Linux: `curl -fsSL https://chatgpt.com/codex/install.sh | sh`	Run `codex`, sign in with ChatGPT
Cursor, Windsurf, VS Code	Install the editor and sign in	Open rea-work, use Agent mode
Claude Desktop	Install from claude.ai/download	Quit it fully before running setup
OpenCode	`npm install --global opencode-ai`	Run `opencode`, then /connect

REA's setup supports many other clients too, including Gemini CLI, GitHub Copilot CLI, Devin, Antigravity and Qwen Code. Each has a **setup ID** you can pass to the setup command.

Client	Setup ID	Client	Setup ID
Claude Code	`claude_code`	Windsurf	`windsurf`
Claude Desktop	`claude_desktop`	VS Code	`vscode`
Codex	`codex`	OpenCode	`opencode`
Cursor	`cursor`	Gemini CLI	`gemini_cli`

3 Connect REA

Adds the MCP connection and investigation skill to your agent.

Option A: let your agent do it. Paste this into your agent's chat:

> **YOUR CODING AGENT**
> *› Install REA and connect it to this coding agent using npx rea-agents@latest setup. Show me the setup plan for approval, then verify the installation.*

Option B: run it yourself.

```
TERMINAL
npx rea-agents@latest setup

# or target one client directly
npx rea-agents@latest setup --client claude_code
```

- If npm asks to download the package, type **y** and press Enter.
- In the client list, use the arrow keys to move, Space to select, Enter to continue.
- Review the plan before approving. REA backs up your existing configuration first.
- You can **skip the optional Hopper install** for your first run.

Restart your agent. For desktop apps, quit completely (⌘ Q on macOS, or quit from the tray on Windows). Closing the window isn't enough. For terminal agents, exit and start again.

Check the connection. Replace the client ID with yours:

```
TERMINAL
npx -y rea-agents@latest doctor --client claude_code --json
```

A healthy setup reports `"healthy": true`. Then confirm from the agent's side:

> **YOUR CODING AGENT**
> *› Check that REA is connected and show me which REA tools you can use.*

4 Run your first investigation

A tiny Electron notes app with an Export button, built for this exercise.

The example app has six files. Your agent inspects the code without installing or launching Electron. Ask:

> **YOUR CODING AGENT**
> *› Download https://rea.tools/examples/notes-example.zip into a new example folder and unpack it. Use REA to find what happens when I click Export notes. Show where notes.csv is written, which fields it contains, and the relevant files and lines.*

What a good answer looks like: the button sends a `notes:export` message, the main process formats notes as CSV and writes `notes.csv` to your Downloads folder, and the columns are `id` and `title`. Each claim should come with file and line references.

If your agent can't download files

Download the ZIP from rea.tools/examples/notes-example.zip yourself, unzip it, and give your agent the full path to the notes-example folder with the same question. To copy a path: on Windows right-click the folder and choose *Copy as path*; on macOS hold Option while right-clicking and choose *Copy as Pathname*; on any system, drag the folder into a terminal.

5 Inspect a real Electron app

Most Electron apps keep their code in a single app.asar file.

Find the app.asar file, then copy it into your working folder so you never touch the installed app:

System	Usual location
Windows	`C:\Users\<you>\AppData\Local\Programs\<App>\resources\app.asar`
macOS	`/Applications/<App>.app/Contents/Resources/app.asar (right-click the app, Show Package Contents)`
Linux	`/opt/<App>/resources/app.asar`

Quick check from the terminal. This maps the app's modules and connections without running it:

```
TERMINAL
npx -y rea-agents@latest analyze-javascript-application "/absolute/path/to/app.asar" --json
```

Connections REA can't resolve are marked in the result. Large, bundled apps take longer, so run this once before you rely on it. **Then ask your agent plain questions**, for example:

YOUR CODING AGENT

› Use REA to inspect the app at /absolute/path/to/app.asar. Where does it store my data on this computer, and is that storage protected? Show the relevant files and lines.

Good beginner questions:

- What happens when I click a specific button?
- Where does this app save my files and settings?
- When I delete something, is it really removed or just hidden?
- What websites or servers does this app connect to, and why?

Verify the answer

For open-source apps, open the project's GitHub repository and find the code your agent pointed to. AI agents can be confidently wrong, so checking two or three references builds real trust in the result.

GOING FURTHER

Other kinds of software

REA can investigate much more than JavaScript, with the right local tool (a **provider**) installed. Each has its own guide on rea.tools.

Target	Extra tools needed
Native programs and libraries	Ghidra (free, needs JDK 21+), Hopper (macOS/Linux) or IDA
Websites	Chrome, Chromium or Edge
Android APK	JADX and JDK 17+
.NET, Flutter, packages	Node.js and npm
Network captures (HAR, PCAP)	Node.js or mitmdump; TShark on Linux
WASM, firmware, SQLite, EVM	WABT, Binwalk/Unblob, Node.js 24.15+, depending on target

Start simple
Native binaries through Ghidra need more setup and produce harder-to-read output. Get comfortable with an Electron app first.

Troubleshooting

Problem	Fix
Agent doesn't see REA	Restart the client fully, then run the doctor check. In editors, enable REA in MCP settings and allow its tools.
“Scripts disabled” on Windows	Use npx.cmd and npm.cmd instead of npx and npm.
Agent runs inside WSL	Install Node.js and REA inside WSL and use Linux paths there.
Path has spaces	Put the path in quotes in the terminal.
A provider fails	Run `npx -y rea-agents@latest doctor --provider ghidra --json`. The report names the failed check and the next step.
Old Node.js from Linux packages	Distribution packages are often older than 22.19. Check `node -v` or use fnm.

Update or remove

TERMINAL

```
# refresh the connection (review the plan, then restart your agent)
npx rea-agents@latest setup

# remove REA's client configuration and skill
npx rea-agents@latest uninstall

# optional: a permanent rea command, updated with `rea update`
npm install --global rea-agents
```

Useful links

Resource	Link
Setup guide	`rea.tools/get-started`
Investigation guides	`rea.tools/guides`
Source code and issues	`github.com/morluto/rea`
Community	`discord.gg/GkcryMnJDM`

Version notes: this guide reflects the npm 6.4.0 release and the setup page as of 25 October 2026. Commands and requirements change, so check rea.tools/get-started if something doesn't match.